Problem statement and feature requirements

1. Problem statement

Teams and users need a **real-time messaging backend** that can:

- Register users and discover who else exists for starting conversations.
- Organize people into **groups** with an admin who can manage membership and title.
- Exchange **group** and **direct (1:1)** messages, with history available over HTTP and live delivery over WebSockets.
- Surface **who is online** at user and group scope, and broadcast presence changes to connected clients.
- Record **read receipts** for messages and optionally notify clients in real time.
- Generate **in-app notifications** when peers receive new messages through the HTTP API paths that trigger them.

The codebase solves this with an **Express + Sequelize (MySQL) + Socket.IO** service (`src/server.ts`, `src/app.ts`), using a simple **`x-user-id` header** as the acting user for authenticated-style routes (no JWT in tree at time of writing).

---

2. Cross-cutting execution requirements (from `src`)

Feature	Behavior	Primary sources
HTTP API prefix	All JSON routes live under `/api/v1`.	`src/app.ts`
Acting user	Routes mounted after `requireUserContext` require header **`x-user-id`** (positive integer). Invalid or missing → `401` JSON error.	`src/routes/index.ts`, `src/middlewares/auth-context.middleware.ts`
Users without header	`POST /users`, `GET /users` work **without** `x-user-id`. `GET /users/directory` adds `requireUserContext` on that route only.	`src/routes/user.routes.ts`, `src/routes/index.ts`
Request validation	Bodies validated with Joi where schemas exist; failures go through validation middleware.	`src/middlewares/validate.middleware.ts`, `src/validators/*.ts`
Errors	Central error middleware; typed `HttpError` for expected failures.	`src/middlewares/error.middleware.ts`, `src/utils/httpError.ts`
Success shape	JSON success wrapper via `successResponse`.	`src/utils/apiResponse.ts`

Feature	Behavior	Primary sources
Database	Sequelize connects and syncs (`alter` in development).	`src/config/database.ts`, `src/server.ts`
Real-time transport	Socket.IO on the same HTTP server; CORS open for `*` in server bootstrap.	`src/server.ts`, `src/socket/io.ts`
Socket auth	Connection rejected if `handshake.auth.user_id` is missing or not a valid number.	`src/socket/register-events.ts`
Graceful shutdown	On `SIGTERM` / `SIGINT`: disconnect sockets, close Socket.IO, close Sequelize, close HTTP server.	`src/server.ts`

3. REST features to execute (HTTP)

Base URL path segment: **`/api/v1`**. Unless noted, assume **`x-user-id`** is required (see §2).

3.1 Users (`src/routes/user.routes.ts`, `src/controllers/user.controller.ts`, `src/services/user.service.ts`)

Method	Path	Auth	Execution
`POST`	`/users`	No	Create user: `first_name`, `last_name`, `email_id` (Joi: `src/validators/user.validator.ts`). **201** with created user.
`GET`	`/users`	No	List all users. **200**.
`GET`	`/users/directory`	**Yes** (`x-user-id`)	Paginated directory for DM picking: query `page` (≥1), `limit` (default 10, max 50). Excludes current user. **200**.

3.2 Groups (`src/routes/group.routes.ts`, `src/controllers/group.controller.ts`, `src/services/group.service.ts`)

Method	Path	Execution
`GET`	`/groups/my`	List groups for `x-user-id`. **200**.
`POST`	`/groups`	Body: `group_title`, optional `user_ids[]`. Creator is admin; creates group and membership. **201**.
`PUT`	`/groups/:group_id`	Body: `group_title`. Admin-only update. **200**.
`GET`	`/groups/:group_id/members`	List members. **200**.

Method	Path	Execution
POST	/groups/:group_id/members	Body: user_ids (non-empty). Admin adds members. **200**.
DELETE	/groups/:group_id/members/:user_id	Admin removes member (service rules apply). **200**.
POST	/groups/:group_id/admin/:user_id	Current admin transfers admin to user_id. **200**.

3.3 Chat (src/routes/chat.routes.ts, src/controllers/chat.controller.ts, src/services/chat.service.ts)

Method	Path	Execution
GET	/chat/groups/:group_id/messages	Sender must be group member; up to **500** messages ordered by sent_at. **200**.
POST	/chat/groups/:group_id/messages	Body: message_text (1–2000 chars). Persists, emits **group:message** to room group:{group_id}, creates **notifications** for other members via NotificationService. **201**.
GET	/chat/groups/:group_id/online	Ensures membership; returns online_count and online_user_ids from presenceService. **200**.
GET	/chat/direct/:peer_user_id/messages	Direct thread between x-user-id and peer; **500** message cap. **200**.
POST	/chat/direct/:peer_user_id/messages	Body: message_text. Persists, emits **direct:message** to both user rooms, creates **notification** for peer. **201**.
GET	/chat/direct/:peer_user_id/online	Returns is_online for peer. **200**.
POST	/chat/messages/:message_id/seen	Idempotent seen row; emits **message:seen** globally with seen_users; **200**.

3.4 Notifications (src/routes/notification.routes.ts, src/controllers/notification.controller.ts, src/services/notification.service.ts)

Method	Path	Execution
GET	/notifications	Query: page, limit (default 10, max 50), status = all | read | unread. Paginated list for x-user-id. **200**.

Method	Path	Execution
POST	/notifications/:ntf_id/read	Marks notification read for owner. **200**.

4. Real-time features to execute (Socket.IO)

Implemented in **src/socket/register-events.ts** with **src/services/presence.service.ts**, **src/services/group.service.ts**, **src/services/chat.service.ts**.

Feature	Trigger / event	Execution
Connect	Client connects with auth.user_id	Join user:{user_id}; register socket in presence; **user:presence** broadcast (is_online: true). Load user's groups, join group:{group_id} each, update group presence counts, emit **group:presence** to each group room.
Group message (socket)	Client → **chat:group:send** { group_id, message_text }	ChatService.sendGroupMessage (membership enforced), then server → room **group:message** payload. **Note:** this path does **not** enqueue notifications (notifications are wired on HTTP send in ChatController).
Direct message (socket)	Client → **chat:direct:send** { receiver_user_id, message_text }	ChatService.sendDirectMessage, then **direct:message** to sender and receiver user rooms.
Disconnect	Socket disconnect	Remove socket from presence; **user:presence** with updated online flag; per group **group:presence** with new online_cnt.

Server → client event names to implement on clients: **user:presence**, **group:presence**, **group:message**, **direct:message**, **message:seen** (from HTTP markSeen via io.emit in ChatController).

5. Environment and runtime (src/config/env.ts)

Variable	Role
NODE_ENV	Drives Sequelize sync({ alter }) in development.
PORT	HTTP listen port (default **5000**).
DB_* / DB_SOCKET_PATH	MySQL connection for Sequelize.

6. Source map (where each capability lives)

Area	Key files

Area	Key files
Entry / lifecycle	`src/server.ts`
Express app	`src/app.ts`
Route mounting & auth gate	`src/routes/index.ts`
Users	`src/routes/user.routes.ts`, `src/controllers/user.controller.ts`, `src/services/user.service.ts`, `src/models/user.model.ts`
Groups	`src/routes/group.routes.ts`, `src/controllers/group.controller.ts`, `src/services/group.service.ts`, `src/models/chat-group.model.ts`, `src/models/group-user-map.model.ts`
Chat & seen	`src/routes/chat.routes.ts`, `src/controllers/chat.controller.ts`, `src/services/chat.service.ts`, `src/models/chat-message.model.ts`, `src/models/message-seen.model.ts`
Notifications	`src/routes/notification.routes.ts`, `src/controllers/notification.controller.ts`, `src/services/notification.service.ts`, `src/models/notification.model.ts`
Presence	`src/services/presence.service.ts`
Socket.IO	`src/socket/io.ts`, `src/socket/register-events.ts`
Shared middleware/util	`src/middlewares/*`, `src/utils/*`